

ML_Optimization_Problem2

2025-12-03

STSCI 6520 Homework 2 - Problem 2

K.CHAN

Gaussian Mixture Models, K-Means, and EM Algorithm

- The Expectation-Maximization (EM) algorithm is an iterative method to find maximum likelihood estimates when data is missing or hidden.
- In a Gaussian Mixture Model (GMM), the “hidden” data is the knowledge of which Gaussian component generated which data point.
- The Algorithm Steps: Initialization: Start with initial guesses for weights (w), means (μ), and covariances (Σ).
- In this problem, we use the output of K-means for this. E-Step (Expectation): Calculate the “responsibility” γ_{ik} , which is the probability that point i belongs to cluster k , given the current parameters.

$$\gamma_{ik} = \frac{w_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^3 w_j \cdot \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

- M-Step (Maximization): Update the parameters to maximize the likelihood using the responsibilities from the E-step.
- New Weights: $w_k^{new} = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}$
- New Means: $\mu_k^{new} = \frac{\sum_{i=1}^n \gamma_{ik} x_i}{\sum_{i=1}^n \gamma_{ik}}$
- New Covariances: $\Sigma_k^{new} = \frac{\sum_{i=1}^n \gamma_{ik} (x_i - \mu_k^{new})(x_i - \mu_k^{new})^T}{\sum_{i=1}^n \gamma_{ik}}$ Repeat until convergence.

```
set.seed(8888)
#1. data gen
# Parameters
true_weights <- c(0.6, 0.2, 0.2)
mu <- list(
  c(0, 0),
  c(-1.6, 1.4),
  c(1.6, 1.4)
)
sigma <- list(
  diag(1, 2),
  diag(0.25, 2),
  diag(0.25, 2)
)
n <- 200
```

```

# Function to generate data
generate_gmm_data <- function(n, weights, means, sigmas) {
  # Sample cluster assignments based on weights
  z <- sample(1:3, n, replace = TRUE, prob = weights)

  X <- matrix(0, nrow = n, ncol = 2)
  for (i in 1:n) {
    k <- z[i]
    X[i, ] <- rmvnorm(1, mean = means[[k]], sigma = sigmas[[k]])
  }

  return(list(X = X, labels = z))
}

data <- generate_gmm_data(n, true_weights, mu, sigma)
X <- data$X
true_labels <- data$labels

# Plot Setup
plot_clusters <- function(X, labels, title) {
  colors <- c("red", "blue", "green")
  plot(X, col = colors[labels], pch = 19, main = title,
        xlab = "X1", ylab = "X2", asp = 1)
  grid()
}
#plot_clusters(X, true_labels, "True Cluster Assignments")

```

```

# Since K-means/EM are unsupervised, Cluster 1 might assume the ID "2".
# We must permute predicted labels to match ground truth to minimize error.
get_best_match_error <- function(true_labs, pred_labs) {
  library(combinat) # For permn, or we hardcode the 6 permutations for 3 clusters

  perms <- list(
    c(1, 2, 3), c(1, 3, 2),
    c(2, 1, 3), c(2, 3, 1),
    c(3, 1, 2), c(3, 2, 1)
  )

  best_error <- 1.0
  best_mapping <- NULL

  for (p in perms) {
    # Map prediction k -> p[k]
    mapped_preds <- p[pred_labs]
    error <- mean(mapped_preds != true_labs)

    if (error < best_error) {
      best_error <- error
      best_mapping <- mapped_preds
    }
  }

  return(list(error = best_error, mapped_labels = best_mapping))
}

```

```
}
```

2. K-Means Implementation

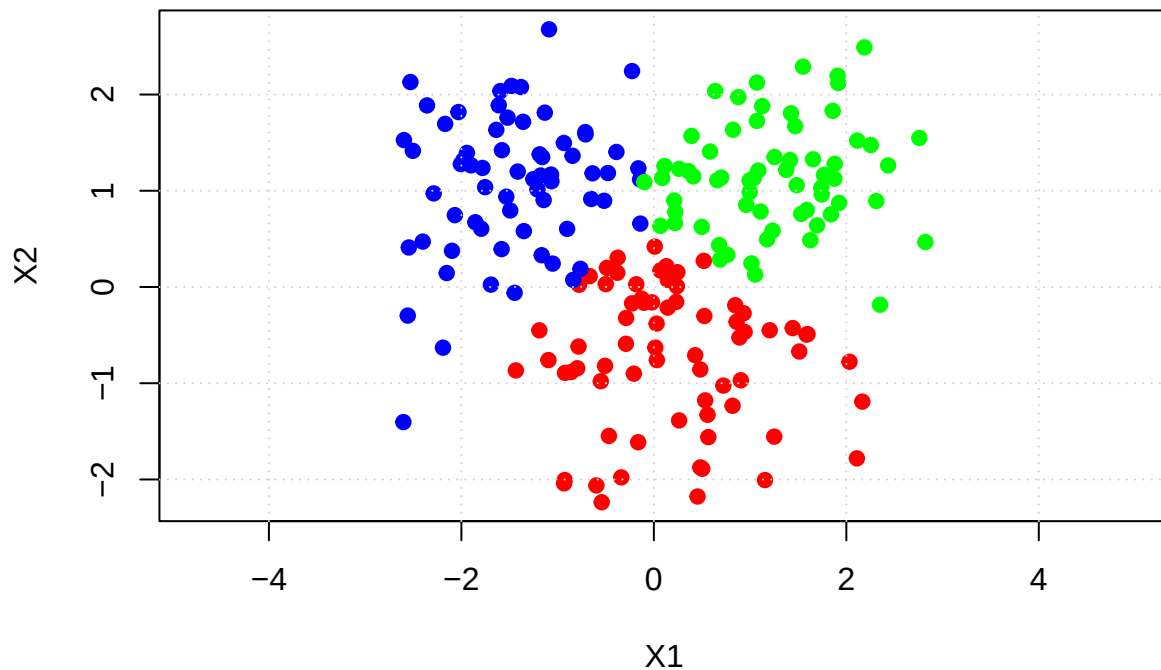
```
run_kmeans <- function(X, K = 3, max_iter = 100) {  
  n <- nrow(X)  
  centroids <- X[sample(1:n, K), ] #sample RANDOMIZES the points.  
  assignments <- rep(0, n)  
  
  for (iter in 1:max_iter) {  
    # Assignment Step  
    dists <- matrix(0, n, K)  
    for (k in 1:K) {  
      # Squared Euclidean distance  
      dists[, k] <- rowSums((sweep(X, 2, centroids[k, ])) ^ 2)  
    }  
    new_assignments <- apply(dists, 1, which.min)  
  
    # Check convergence  
    if (all(assignments == new_assignments)) break  
    assignments <- new_assignments  
  
    # Update Step  
    for (k in 1:K) {  
      if (sum(assignments == k) > 0) {  
        centroids[k, ] <- colMeans(X[assignments == k, , drop = FALSE])  
      }  
    }  
  }  
  return(list(assignments = assignments, centroids = centroids))  
}  
  
km_res <- run_kmeans(X)  
km_eval <- get_best_match_error(true_labels, km_res$assignments)
```

2A and B SOLUTION

- K-Means Fraction of Mistakes: 0.27

```
plot_clusters(X, km_eval$mapped_labels, paste0("K-Means Clustering (Error: ", round(km_eval$error, 3),
```

K-Means Clustering (Error: 0.27)



2c. EM Algorithm Implementation

```
# Initialize EM parameters using K-means results
init_em <- function(X, km_assignments, K = 3) {
  w <- numeric(K)
  mu <- list()
  sigma <- list()

  for (k in 1:K) {
    mask <- (km_assignments == k)
    w[k] <- mean(mask) # Fraction of points in cluster k
    mu[[k]] <- colMeans(X[mask, , drop = FALSE])
    sigma[[k]] <- cov(X[mask, , drop = FALSE])

    # Regularize covariance to avoid singularity if cluster is too small
    if (sum(mask) < 2) sigma[[k]] <- diag(1, 2)
  }
  return(list(w = w, mu = mu, sigma = sigma))
}

run_em <- function(X, init_params, K = 3, max_iter = 100, tol = 1e-6) {
  n <- nrow(X)
  w <- init_params$w
```

```

mu <- init_params$mu
sigma <- init_params$sigma

log_lik_hist <- c()

for (iter in 1:max_iter) {
  # --- E-Step ---
  # Calculate responsibilities gamma_ik
  gammas <- matrix(0, n, K)
  for (k in 1:K) {
    gammas[, k] <- w[k] * dmvnorm(X, mean = mu[[k]], sigma = sigma[[k]])
  }

  # Normalize rows to sum to 1
  row_sums <- rowSums(gammas)
  log_lik <- sum(log(row_sums)) # Track log likelihood
  gammas <- gammas / row_sums

  # Check convergence
  if (iter > 1 && abs(log_lik - tail(log_lik_hist, 1)) < tol) break
  log_lik_hist <- c(log_lik_hist, log_lik)

  # --- M-Step ---
  N_k <- colSums(gammas) # Effective number of points in each cluster

  for (k in 1:K) {
    # Update weights
    w[k] <- N_k[k] / n

    # Update means
    # Weighted sum of X
    mu[[k]] <- colSums(gammas[, k] * X) / N_k[k]

    # Update covariances
    # Weighted covariance matrix
    # (X - mu_k)
    diff <- sweep(X, 2, mu[[k]])
    # Manually compute weighted outer product sum
    # Sigma_k = sum(gamma_ik * (x_i - mu_k)(x_i - mu_k)^T) / N_k
    sig_sum <- matrix(0, 2, 2)
    for (i in 1:n) {
      sig_sum <- sig_sum + gammas[i, k] * (diff[i, ] %*% t(diff[i, ]))
    }
    sigma[[k]] <- sig_sum / N_k[k]
  }
}

# Final hard assignments for error calculation
final_assignments <- apply(gammas, 1, which.max)

return(list(w = w, mu = mu, sigma = sigma,
            assignments = final_assignments, gammas = gammas))
}

```

2C SOLUTION

```
# Running EM Func above  
# Note: Func using the *raw* K-means assignments for initialization, not the mapped ones.  
# The algorithm will adapt the parameters to fit the data distribution.  
init_params <- init_em(X, km_res$assignments)  
em_res <- run_em(X, init_params)  
  
# Evaluate EM Error  
em_eval <- get_best_match_error(true_labels, em_res$assignments)
```

EM Algorithm Fraction of Mistakes: 0.13 Compare Estimated Parameters to True Parameters — Parameter Comparison — True Weights: 0.6 0.2 0.2 Est. Weights: 0.22 0.24 0.53

EM improves performance of k-means algorithm significantly!

```
plot_clusters(X, em_eval$mapped_labels, paste0("EM Clustering (Error: ", round(em_eval$error, 3), ")"))
```

